

软件过程改进方案设计文档

1 改进方案整体概述

1.1 改进目标

- 在不破坏原有ISO/IEC轻量化剪裁流程的前提下，嵌入生成式AI辅助节点，压缩文档、编码、测试重复工作量；
- 弥补4人小型团队无专职架构、专职测试、专职运维的人员短板，降低技术门槛；
- 统一AI输出交付物质量标准，建立“AI生成+人工校验”双控机制，避免模型错误带来项目风险；
- 新增AI过程度量指标，量化AI对项目效率、缺陷率、交付周期的提升效果；
- 保留原有轻量化、快速迭代特性，不新增复杂审批、审计流程，适配2个月短周期学生项目。

1.2 改进核心思路

以ISO/IEC 12207三大过程框架为基底，融合AI4SE技术，遵循「保留核心、嵌入辅助、人工终审、轻量化落地」四大原则：

- 不删除原有标准定义的核心业务活动（需求、设计、编码、测试、运维）；
- 在各环节前置/后置新增AI辅助执行节点，AI仅产出初稿、参考方案，人工负责评审、修正、定稿；
- 新增轻量化AI辅助角色，不增加全职人员，由现有团队兼职兼任；
- 配套轻量化AI工具链，免费/低门槛工具为主，无需服务器部署、付费平台；
- 建立AI风险应对机制，针对模型幻觉、代码漏洞、数据泄露等问题设置流程约束。

1.3 改进覆盖范围

覆盖匹配报告筛选的5大核心环节：

- 开发过程-需求分析
- 开发过程-系统概要/详细设计
- 开发过程-编码实现
- 支持过程-质量保证（测试验证）
- 基本过程-上线维护过程

2 原有角色重构与新增AI相关角色职责

原团队角色：产品负责人、前端开发、后端开发、测试兼文档
无需新增全职人员，由现有人员兼职AI相关岗位，职责拆分如下：

2.1 兼职AI需求分析师（产品负责人兼任）

- 使用LLM工具导入师生需求、访谈记录，生成需求初稿、用户故事；
- 校验AI输出需求文档，修正矛盾、模糊、脱离校园业务的内容；
- 执行AI需求一致性校验，标记模型识别遗漏的师生核心诉求；
- 制定AI需求交付物验收标准。

2.2 兼职AI架构设计工程师（前后端开发轮值兼任）

- 1. 输入需求文档至AI设计工具，生成数据库ER图、接口规范、分层架构方案；
- 2. 评审AI输出数据表、接口逻辑，删除冗余字段、修正不合理外键与查询逻辑；
- 3. 利用AI生成Mermaid流程图、UML用例图，统一设计文档绘图规范。

2.3 兼职AI代码审查员（前后端开发互查）

- 1. 编码阶段开启AI代码补全、静态扫描工具；
- 2. 接收AI代码审查报告，修复模型识别的漏洞、不规范代码；
- 3. 控制AI代码生成比例，核心业务逻辑人工手写，AI仅生成CRUD、基础工具类代码。

2.4 兼职AI测试分析师（测试文档岗兼任）

- 1. 导入需求、接口文档，通过AI批量生成功能、边界、兼容性测试用例；
- 2. 补充AI遗漏的校园业务特殊场景用例（课表批量导入、多社团同时报名等）；
- 3. 利用AI自动分类试用反馈，统计高频缺陷，输出AI辅助测试报告。

2.5 兼职AI运维监控员（全体轮值）

- 1. 使用AI日志分析工具每日扫描服务器报错、小程序访问异常；
- 2. 通过文本AI分类用户反馈，区分BUG、优化需求、新功能诉求；
- 3. 依据AI反馈统计结果，协助产品负责人制定迭代排期。

3 各阶段AI融合改造流程设计

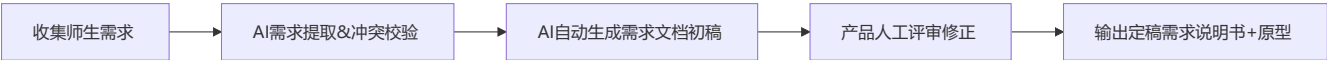
3.1 开发过程-需求分析环节改造流程

原流程

师生需求收集 → 人工整理需求清单 → 绘制原型 → 人工编写需求说明书

改造后完整流程（新增AI节点）

- 1. 产品负责人收集师生文字/问卷反馈；
- 2. **AI辅助节点1**：导入大模型，自动提取功能/非功能需求、剔除重复诉求、标记矛盾需求；
- 3. **AI辅助节点2**：基于整理后的需求自动生成轻量化需求说明书初稿、用户故事；
- 4. 人工评审修正AI输出文档，补充校园特有业务约束；
- 5. 输出定稿需求说明书、产品原型。



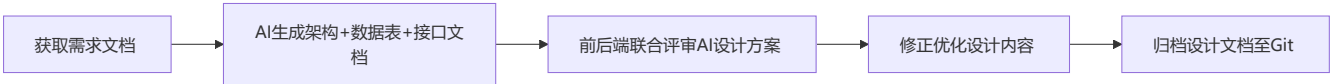
3.2 开发过程-系统设计环节改造流程

原流程

阅读需求文档 → 人工设计数据库 → 手写接口文档 → 手绘ER图

改造后完整流程

- 1. 开发人员获取定稿需求说明书；
- 2. **AI辅助节点**：输入技术栈（微信小程序+Java+MySQL），AI输出分层架构、数据库表结构、接口入参出参、Mermaid ER图；
- 3. 前后端共同评审AI设计方案，修正不合理存储、高耗时查询逻辑；
- 4. 定稿接口文档、数据库设计文档，同步至Git。



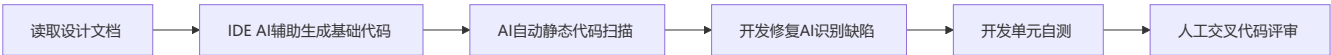
3.3 开发过程-编码实现环节改造流程

原流程

阅读设计文档 → 人工编写代码 → 开发自测 → 人工交叉代码审查

改造后完整流程

- 1. 开发人员拉取最新设计文档；
- 2. **AI实时辅助节点**：IDE内置代码大模型，注释驱动生成基础页面、接口、SQL；
- 3. 编码完成后触发**AI静态代码审查**，输出漏洞、冗余代码整改清单；
- 4. 开发人员修复AI标记问题，完成单元自测；
- 5. 团队交叉人工复核核心业务代码。



3.4 支持过程-质量保证（测试验证）改造流程

原流程

测试阅读需求+接口文档 → 手动编写测试用例 → 执行测试 → 人工整理师生试用反馈

改造后完整流程

- 1. 测试人员获取需求、接口定稿文档；
- 2. **AI辅助节点**：AI批量生成正向、反向、边界场景测试用例表格；
- 3. 测试人员补充校园专属特殊场景用例；
- 4. 执行手工测试，记录缺陷；
- 5. **AI辅助节点**：导入师生试用反馈文本，自动分类BUG/优化建议，统计高频问题；

6. 输出标准化测试报告。



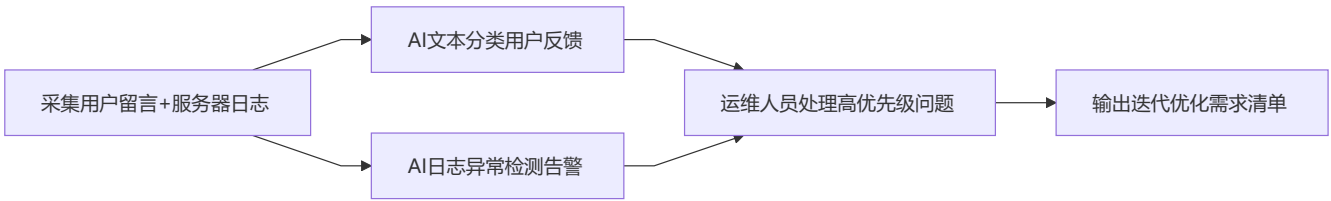
3.5 基本过程-维护运维环节改造流程

原流程

收集小程序用户留言 → 人工分类反馈 → 手动查看服务器日志排查故障

改造后完整流程

- 1. 小程序上线后每日收集用户留言、服务器运行日志；
- 2. **AI辅助节点1**：文本大模型自动分类反馈，统计各类问题占比；
- 3. **AI辅助节点2**：日志AI工具扫描异常接口、访问峰值、报错堆栈；
- 4. 轮值人员查看AI分析结果，优先处理高频率BUG、线上告警；
- 5. 产品依据AI统计数据制定版本迭代计划。



4 AI工具链选型、集成方式与预期提升指标

4.1 需求分析阶段工具

工具名称	核心功能	与现有过程集成方式	预期效率提升指标
豆包大模型	需求提取、需求文档自动生成、矛盾点识别	复制粘贴师生反馈文本，输出文档初稿后导入Word轻量化手册	需求整理耗时降低85%，需求模糊问题减少40%
Notion AI	在线轻量化需求文档自动排版	同步团队共享文档库，直接嵌入项目知识库	文档排版、格式统一工作量清零

4.2 系统设计阶段工具

工具名称	核心功能	与现有过程集成方式	预期效率提升指标
通义灵码AI设计模块	数据库表、分层架构、接口规范生成	复制需求文档至工具，导出Mermaid图表粘贴至设计文档	设计绘图、手写接口文档工作量降低70%
Apifox AI助手	自动生成接口文档、Mock数据	对接后端工程，AI生成接口后直接导入Apifox管理	接口设计调试周期缩短50%

4.3 编码实现阶段工具

工具名称	核心功能	与现有过程集成方式	预期效率提升指标
GitHub Copilot	代码补全、CRUD自动生成、小程序页面代码	VS Code插件内置，编码全程实时调用	基础业务编码速度提升45%
通义灵码静态扫描	漏洞检测、代码规范重构	IDE内置，提交代码前一键扫描	基础代码缺陷数量下降35%

4.4 测试验证阶段工具

工具名称	核心功能	与现有过程集成方式	预期效率提升指标
Testin XAgent	AI自动生成功能、边界测试用例	导入需求文档，导出Excel用例表格	测试用例编写耗时降低90%，场景覆盖度提升30%
豆包文本分类提示词	试用反馈自动归类统计	批量导入用户留言，输出分类统计表格	反馈整理时间减少92%

4.5 运维维护阶段工具

工具名称	核心功能	与现有过程集成方式	预期效率提升指标
阿里云SLS智能日志分析	日志异常自动识别、报错根因提示	服务器日志同步至SLS控制台，每日AI自动巡检	日志故障排查耗时降低80%
豆包文本分类模型	线上反馈自动归类、热度统计	定期导出小程序评论批量处理	迭代需求梳理效率提升85%

5 AI输出物质质量标准与人工校验机制

所有AI生成内容**禁止直接定稿上线**，强制执行「AI初稿输出→人工校验修改→归档定稿」三级管控，各交付物质量标准如下：

5.1 AI需求文档输出质量标准

- 1. 必须完整覆盖五大业务模块基础功能，无功能缺失；
- 2. 功能性/非功能性需求划分清晰，无重复、矛盾描述；
- 3. 适配校园场景，不存在脱离师生使用场景的虚构需求；
- 4. 人工修改内容占比不低于10%，留存修改记录。

5.2 AI设计文档（数据库/接口）质量标准

- 1. 数据表无冗余字段、无不合理外键关联；
- 2. 接口入参、出参符合微信小程序前后端交互规范；
- 3. 高并发查询场景（课表、公告）具备缓存优化方案；

4. 开发人员评审签字确认后方可归档Git。

5.3 AI生成代码质量标准

1. AI仅允许生成基础CRUD、页面布局、SQL语句，核心业务判断逻辑必须人工编写；
2. AI扫描标记的漏洞、安全风险必须100%修复；
3. 代码命名、注释符合团队统一规范，AI生成不规范代码全部重构。

5.4 AI测试用例输出质量标准

1. AI输出用例覆盖正常流程、边界值、异常输入三大场景；
2. 测试人员必须补充校园特有业务场景用例（占总用例15%以上）；
3. 用例步骤清晰、预期结果明确，无模糊描述。

5.5 AI运维分析报告质量标准

1. 用户反馈分类准确率 $\geq 90\%$ ，人工修正错误分类；
2. AI告警日志需人工复现验证，排除模型误报；
3. 统计数据作为迭代参考，最终排期由产品人工决策。

6 AI引入风险评估与应对策略

6.1 风险1：模型幻觉，AI输出错误需求、错误设计、错误代码

- 风险等级：高
- 风险表现：AI编造不存在的校园功能、生成存在漏洞的SQL、错误接口逻辑；
- 应对策略：流程强制设置人工终审节点，所有AI产出必须经过对应岗位人员评审；核心业务代码禁止完全由AI生成；留存AI原始输出与人工修改对比记录。

6.2 风险2：过度依赖AI，降低团队开发、需求分析自主能力

- 风险等级：中
- 风险表现：开发人员完全依靠AI写代码，缺乏独立排错、设计能力；产品完全依赖AI梳理需求；
- 应对策略：明确AI使用规范，核心业务逻辑、架构方案必须人工主导；每周例会安排无AI辅助小型编码、需求梳理练习。

6.3 风险3：数据安全风险，项目需求、代码、用户数据上传第三方AI平台泄露

- 风险等级：中
- 风险表现：将数据库结构、小程序业务代码、校内师生隐私需求粘贴至公有大模型；
- 应对策略：脱敏处理数据，隐藏学号、姓名等隐私信息；不直接上传完整后端业务代码，仅粘贴片段；优先使用支持本地离线的代码大模型。

6.4 风险4：AI工具限制，免费模型存在长度、并发、调用次数限制

- 风险等级：低
- 风险表现：项目高峰期批量生成文档、用例时模型输出截断、调用失败；
- 应对策略：拆分输入内容，分段生成；提前人工储备基础模板，AI工具不可用时人工兜底完成。

6.5 风险5：AI输出交付物格式混乱，增加整理成本

- 风险等级：低
- 风险表现：AI生成文档排版杂乱、表格缺失、图表无法直接复用；
- 应对策略：统一标准化提示词模板，固定输出Markdown/表格格式；制定统一轻量化文档模板，AI内容直接填充模板。

7 方案落地实施计划

1. **第1周**：完成AI工具安装、提示词模板标准化，梳理各环节AI输入输出模板；
2. **第2周**：按流程改造方案更新原有ISO/IEC软件过程文档，绘制全流程AI融合流程图；
3. **第3周**：选取测试环节完成原型验证，记录人工/AI对比数据，撰写验证报告；
4. **第4周**：整合全部流程、角色、工具、风险内容，完成最终文档交付。

8 方案总结

本改进方案以原有ISO/IEC 12207剪裁后的校园小程序软件过程为基底，针对5大低效环节嵌入生成式AI辅助节点，通过兼职复用现有人员设置AI相关岗位，配套轻量化免费AI工具链，明确各交付物人工校验标准，同时完整覆盖AI幻觉、数据安全、技术依赖等风险的管控策略。

整体改造方案贴合4人小型学生团队、2个月短周期、轻量化文档的项目约束，不引入复杂流程与高额工具成本，既保留国际标准过程规范性，又借助AI大幅降低重复人力工作，同时设置完善管控机制规避AI技术缺陷带来的项目质量风险，具备较强落地可行性。

9 参考文献

1. 中国信通院《智能化软件工程技术和应用要求 第3部分：智能测试能力》(2024)
2. 《AI4SE 行业现状调查报告》(2024)
3. Gtest 全球软件测试技术峰会 2025 会议资料
4. 《生成式 AI 在需求工程中的应用》
5. 《AI 原生自动化测试范式研究》